

P0901R10

Size feedback in operator new

Published Proposal, 2022-11-09

This version:

<http://wg21.link/P0901R10>

Authors:

[Andrew Hunter](#)

[Chris Kennelly](#) (Google)

[Thomas Köppe](#) (Google DeepMind)

Audience:

CWG, EWG, LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Provide access to actual malloc buffer sizes for users.

Table of Contents

1 Motivation

- 1.1 nallocx: not as awesome as it looks
 - 1.1.1 nallocx must give a conservative answer
 - 1.1.2 nallocx duplicates work
 - 1.1.3 nallocx hides information from malloc
- 1.2 after allocation is too late
- 1.3 realloc's day has passed

2 Proposal

- 2.1 Allocation functions

2.2 *New-expressions*

3 **Proposed Wording**

4 **Alternative Designs Considered**

4.1 Parameters and return types

4.2 New expressions

4.2.1 Interaction of allocation functions and *new-expressions*

4.2.2 Which kind of "size"

5 **Discussion**

5.1 How many `::operator new`'s?

5.2 Implementation difficulty

5.3 Interaction with Sized Delete

5.4 Advantages

5.5 Naming

6 **Related work**

7 **History**

7.1 R9 → R10

7.2 R8 → R9

7.3 R7 → R8

7.4 R6 → R7

7.5 R5 → R6

7.6 R4 → R5

7.7 R3 → R4

7.8 R2 → R3

7.9 R1 → R2

7.10 R0 → R1

References

Informative References

§ 1. Motivation

Throughout this document, "malloc" refers to the **implementation** of `::operator new` both as fairly standard practice for implementers, and to make clear the distinction between the interface and the implementation.

Everyone's favorite dynamic data structure, `std::vector`, allocates memory with code that looks something like this (with many details, like `Allocator`, templating for non `char`, and exception safety, elided):

```
void vector::reserve(size_t new_cap) {
    if (capacity_ >= new_cap) return;
    const size_t bytes = new_cap;
    void *newp = ::operator new(new_cap);
    memcpy(newp, ptr_, capacity_);
    ptr_ = newp;
    capacity_ = bytes;
}
```

Consider the sequence of calls:

```
std::vector<char> v;
v.reserve(37);
// ...
v.reserve(38);
```

All reasonable implementations of malloc round sizes, both for alignment requirements and improved performance. It is extremely unlikely that malloc provided us exactly 37 bytes. We do not need to invoke the allocator here...except that we don't know that for sure, and to use the 38th byte would be undefined behavior. We would like that 38th byte to be usable without a roundtrip through the allocator.

This paper proposes an API making it safe to use that byte, and explores many of the design choices (not all of which are obvious without implementation experience.)

§ 1.1. nallocx: not as awesome as it looks

The simplest way to help here is to provide an informative API answering the question "If I ask for N bytes, how many do I actually get?" [jemalloc](#) and [TCMalloc](#) call this `nallocx`. We can then use

that hint as a smarter parameter for operator new:

```
void vector::reserve(size_t new_cap) {
    if (capacity_ >= new_cap) return;
    const size_t bytes = nallocx(new_cap, 0);
    void *newp = ::operator new(bytes);
    memcpy(newp, ptr_, capacity_);
    ptr_ = newp;
    capacity_ = bytes;
}
```

This is a good start, and does in fact work to allow vector and friends to use the true extent of returned objects. But there are three significant problems with this approach.

§ 1.1.1. nallocx must give a conservative answer

While many allocators have a deterministic map from requested size to allocated size, it is by no means guaranteed that all do. Presumably they can make a reasonably good guess, but if two calls to `::operator new(37)` might return 64 and 128 bytes, we'd definitely rather know the right answer, not a conservative approximation.

§ 1.1.2. nallocx duplicates work

Allocation is often a crucial limit on performance. Most allocators compute the returned size of an object as part of fulfilling that allocation...but if we make a second call to `nallocx`, we duplicate all that communication, and also the overhead of the function call.

§ 1.1.3. nallocx hides information from malloc

The biggest problem (for the authors) is that `nallocx` discards information malloc finds valuable (the user's intended allocation size.) That is: in our running example, malloc normally knows that the user wants 37 bytes (then 38), but with `nallocx`, we will only ever be told that they want 40 (or 48, or whatever `nallocx(37)` returns.)

Google's malloc implementation ([\[TCMalloc\]](#)) rounds requests to one of a small (<100) number of *sizeclasses*: we maintain local caches of appropriately sized objects, and cannot do this for every possible size of object. Originally, these sizeclasses were just reasonably evenly spaced among the

range they cover. Since then, we have used extensive telemetry on allocator use in the wild to tune these choices. In particular, as we know (approximately) how many objects of any given size are requested, we can solve a fairly simple optimization problem to minimize the total internal fragmentation for any choice of N sizeclasses.

Widespread use of `mallocx` breaks this. By the time TCMalloc's telemetry sees a request that was hinted by `mallocx`, to the best of our knowledge the user *wants* exactly as many bytes as we currently provide them. If a huge number of callers wanted 40 bytes but were currently getting 48, we'd lose the ability to know that and optimize for it.

Note that we can't take the same telemetry from `mallocx` calls: we have no idea how many times the resulting hint will be used (we might not allocate at all, or we might cache the result and make a million allocations guided by it.) We would also lose important information in the stack traces we collect from allocation sites.

Optimization guided by malloc telemetry has been one of our most effective tools in improving allocator performance. It is important that we fix this issue *without* losing the ground truth of what a caller of `::operator new` wants.

These three issues explain why we don't believe `mallocx` is a sufficient solution here.

§ 1.2. after allocation is too late

Another obvious suggestion is to add a way to inspect the size of an object returned by `::operator new`. Most mallocs provide a way to do this; `[jemalloc]` calls it `salloccx`. Vector would look like:

```
void vector::reserve(size_t new_cap) {
    if (capacity_ >= new_cap) return;
    void *newp = ::operator new(new_cap);
    const size_t bytes = salloccx(newp);
    memcpy(newp, ptr_, capacity_);
    ptr_ = newp;
    capacity_ = bytes;
}
```

This is worse than `mallocx`. It fixes the non-constant size problem, and avoids a feedback loop, but the performance issue is worse (this is the major issue *fixed* by `[SizedDelete]`!), and what's worse, the above code invokes UB as soon as we touch byte `new_cap+1`. We could in principle change the standard, but this would be an implementation nightmare.

§ 1.3. realloc's day has passed

We should also quickly examine why the classic C API `realloc` is insufficient.

```
void vector::reserve(size_t new_cap) {  
    if (capacity_ >= new_cap) return;  
    ptr_ = realloc(ptr_, new_cap);  
    capacity_ = new_cap;  
}
```

In principle a `realloc` from 37 to 38 bytes wouldn't carry the full cost of allocation. But it's dramatically more expensive than making no call at all. What's more, there are a number of more complicated dynamic data structures that store variable-sized chunks of data but are never actually resized. These data structures still deserve the right to use all the memory they're paying for.

Furthermore, `realloc`'s original purpose was not to allow the use of more bytes the caller already had, but to (hopefully) extend an allocation in place to adjacent free space. In a classic `malloc` implementation this would actually be possible...but most modern allocators use variants of slab allocation. Even if the 65th byte in a 64-byte allocation isn't in use, they cannot be combined into a single object; it's almost certainly required to be used for the next 64-byte allocation. In the modern world, `realloc` serves little purpose.

§ 2. Proposal

§ 2.1. Allocation functions

We propose adding new overloads of `::operator new` (so-called "size-returning allocation functions") that directly inform the user of the size available to them. C++ makes `::operator new` replaceable (15.5.4.6), allowing a program to provide its own version different from the standard library's implementation.

The new overloads are selected by a tag argument of type `std::return_size_t`, usually provided as the value `std::return_size`. (This is analogous to `std::nothrow_t/std::nothrow` and to `std::align_val_t`.)

The new overloads return a new type, `std::sized_allocation_t<void>`, which is a specialization of the new class template `std::sized_allocation_t` and which stores a void pointer that corresponds to the pointer returned by the existing allocation functions, as well as the new size feedback information.

§ 2.2. *New-expressions*

Additionally, we propose exposing the size feedback from *new-expressions*, too. We arrive at this design through the following chain of reasoning:

1. We want an allocation function with size feedback (motivated above).
2. Allocation functions in C++ are called `operator new`, so this facility is provided as new overloads of `operator new`. (This is similar to how extended alignment support was added as a set of new overloads.)
3. Allocation functions are generically used by *new-expressions*, either of the "usual" or of "placement" form.
4. Therefore, if new allocation functions are proposed, then there must be matching *new-expressions* that use them. We call these "size-returning *new-expressions*"; they are placement *new-expressions* whose first placement argument is `std::return_size`.

The interesting wrinkle here is that our new allocation functions return a novel type, and not `void*` like the existing functions. Consequently, a size-returning *new-expression* should also have a different type. Whereas `new T` is a `T*`, we propose that `new (std::return_size) T` is a `std::sized_allocation_t<T>`, and has values as follows:

- For `new`, a pointer to the object created and the size of the allocation in bytes.
- For `new[]`, a pointers to the initial element of the array and the size of the allocation in bytes (less an unspecified overhead):

```
auto [p, sz] = new (std::return_size) T[5];
for (int i = 5; i < sz / sizeof(T); i++) {
    new (p[i]) T;
}
for (int i = 0; i < sz / sizeof(T); i++) {
    p[i].DoStuff();
}
for (int i = 5; i < sz / sizeof(T); i++) {
    p[i].~T();
}
delete[] p;
```

- See [§ 4.2.2 Which kind of "size"](#) for a discussion on the meaning of "size".

A size-returning *new-expression* is ill-formed if no suitable allocation function is found through the usual overload resolution, but see [§ 4.2.1 Interaction of allocation functions and new-expressions](#) for a

discussion of alternatives.

§ 3. Proposed Wording

We propose wording, relative to [\[CppDraft\]](#):

- Amend [basic.stc.dynamic.allocation], paragraph 1:

The library provides default definitions for the global allocation and deallocation functions. Some global allocation and deallocation functions are replaceable ([`new.delete`]). A C++ program shall provide at most one definition of a replaceable allocation or deallocation function. Any such function definition replaces the default version provided in the library ([`replacement.functions`]). The following allocation and deallocation functions ([`support.dynamic`]) are implicitly declared in global scope in each translation unit of a program.

```
[[nodiscard]] void* operator new(std::size_t);
[[nodiscard]] void* operator new(std::size_t, std::align_val_t);
[[nodiscard]] std::size_t operator new(std::size_t,
std::return_size_t);
[[nodiscard]] std::size_t operator new(std::size_t, std::align_val_t,
std::return_size_t);

void operator delete(void*) noexcept;
void operator delete(void*, std::size_t) noexcept;
void operator delete(void*, std::align_val_t) noexcept;
void operator delete(void*, std::size_t, std::align_val_t) noexcept;

[[nodiscard]] void* operator new[](std::size_t);
[[nodiscard]] void* operator new[](std::size_t, std::align_val_t);
[[nodiscard]] std::size_t operator new[](std::size_t,
std::return_size_t);
[[nodiscard]] std::size_t operator new[](std::size_t, std::align_val_t,
std::return_size_t);

void operator delete[](void*) noexcept;
void operator delete[](void*, std::size_t) noexcept;
void operator delete[](void*, std::align_val_t) noexcept;
void operator delete[](void*, std::size_t, std::align_val_t) noexcept;
```

These implicit declarations introduce only the function names `operator new`, `operator new[]`, `operator delete`, and `operator delete[]`.

[Note 2: The implicit declarations do not introduce the names `std`, `std::size_t`, `std::align_val_t`, `std::return_size_t` or any other names that the library uses to declare these names. Thus, a *new-expression*, *delete-expression*, or function call that refers to one of these functions without importing or including the header `<new>` is well-formed. However, referring to `std` or `std::size_t` or `std::align_val_t` or `std::return_size_t` is ill-formed unless the name has been declared by importing or including the appropriate header. — end note]

Allocation and/or deallocation functions may also be declared and defined for any class ([class.free]).

- Amend [basic.stc.dynamic.allocation], paragraph 1:

An allocation function that is not a class member function shall belong to the global scope and not have a name with internal linkage. The return type shall be `std::size_allocation_t<void>` ([new.syn]) if the allocation function is a size-returning allocation function and `void*` otherwise. The first parameter shall have type `std::size_t` ([support.types]). The first parameter shall not have an associated default argument ([dcl.fct.default]). The value of the first parameter is interpreted as the requested size of the allocation. An allocation function is a size-returning allocation function if it has a second parameter of type `std::return_size_t`, or it has a second parameter of type `std::align_val_t` and a third parameter of type `std::return_size_t`. An allocation function can be a function template. Such a template shall declare its return type and first parameter as specified above (that is, template parameter types shall not be used in the return type and first parameter type). Allocation function templates shall have two or more parameters as specified above.

- Insert [basic.stc.dynamic.allocation], after paragraph 1:

A size-returning allocation function returns a value with a member subobject that is a pointer suitable to hold the address of potentially allocated storage, and therefore all allocation functions are said to return a pointer value.

- Insert [expr.new], after paragraph 9:

A new-pointer is a pointer that results from evaluating the new-expression:

- If the new-expression is a size-returning placement new-expression, then new-pointer is the ptr member of `std::size_allocation_t<T>` ([new.syn]).
- Otherwise, new-pointer is a prvalue of type "pointer to T".

- Amend [expr.new], paragraph 10:

When the allocated type is "array of N T" (that is, the *noPtr-new-declarator* syntax is used or the *new-type-id* or *type-id* denotes an array type), the *new-expression* yields a prvalue of type "pointer to T" that the *new-pointer* points to the initial element (if any) of the array. Otherwise, let T be the allocated type; the *new-expression* is a prvalue of type "pointer to T" that *new-pointer* points to the object created.

- Amend [expr.new], paragraph 16:

When a *new-expression* calls an allocation function and that allocation has not been extended, the *new-expression* passes the amount of space requested to the allocation function as the first argument of type `std::size_t`. That argument shall be no less than the size of the object being created; it may be greater than the size of the object being created only if the object is an array and the allocation function is not a non-allocating form ([*new.delete.placement*]). For arrays of `char`, `unsigned char`, and `std::byte`, the difference between the result of the *new-expression* *new-pointer* and the address returned by the allocation function shall be an integral multiple of the strictest fundamental alignment requirement of any object type whose size is no greater than the size of the array being created.

- Insert [expr.new], after paragraph 17:

The result of a size-returning placement *new-expression* is an object of type `std::scoped_allocator_t<T>` ([*new.syn*]) whose `ptr` member points to the object created and whose `bytes` member satisfies , where

- `bytes` is the number of bytes of valid storage starting from `ptr`, and
- when the allocated type is "array of N T"
 - requested is `sizeof(T) * N`, and
 - returned is the number of bytes of storage returned by the size-returning allocation function less any array allocation overhead,
- otherwise
 - requested is `sizeof(T)`, and
 - returned is the number of bytes of storage returned by the size-returning allocation function.

- Amend [expr.new], paragraph 19:

[Example 6:

- `new(std::return_size) T` results in one of the following calls :
 - `operator new(sizeof(T), std::return_size)`
 - `operator new(sizeof(T), std::align_val_t(alignof(T)), std::return_size)`
- `new(std::return_size, 2, f) T` results in one of the following calls :
 - `operator new(sizeof(T), std::return_size, 2, f)`
 - `operator new(sizeof(T), std::align_val_t(alignof(T)), std::return_size, 2, f)`
- `new(std::return_size) T[5]` results in one of the following calls :
 - `operator new[](sizeof(T) * 5 + x, std::return_size)`
 - `operator new[](sizeof(T) * 5 + x, std::align_val_t(alignof(T)), std::return_size)`
- `new(std::return_size, 2, f) T[5]` results in one of the following calls :
 - `operator new[](sizeof(T) * 5 + x, std::return_size, 2, f)`
 - `operator new[](sizeof(T) * 5 + x, std::align_val_t(alignof(T)), std::return_size, 2, f)`

Here, each instance of `x` is a non-negative unspecified value representing array allocation overhead; the result of the *new-expression* will be offset by this amount from the value returned by `operator new[]`. This overhead may be applied in all array *new-expressions*, including those referencing a placement allocation function, except when referencing the library function `operator new[](std::size_t, void*)`. The amount of overhead may vary from one invocation of `new` to another. — end example]

- Amend [expr.new], paragraph 21:

[Note 10: Unless an allocation function has a non-throwing exception specification, it indicates failure to allocate storage by throwing a `std::bad_alloc` exception ([basic.stc.dynamic.allocation], [except], [bad.alloc]); ~~it returns~~ new-pointer is a non-null pointer otherwise. If the allocation function has a non-throwing exception specification, ~~it returns~~ new-pointer is null to indicate failure to allocate storage and new-pointer is a non-null pointer otherwise. — end note]

If the allocation function is a non-allocating form ([new.delete.placement]) ~~that returns~~ whose new-pointer null, the behavior is undefined. Otherwise, if the allocation function 's new-pointer is ~~returns~~ null, initialization shall not be done, the deallocation function shall not be called, and the value of the new-expression shall be null.

- Amend [expr.new], paragraph 22:

[Note 11: When the allocation function 's new-pointer is ~~returns~~ a value other than null, it must be a pointer to a block of storage in which space for the object has been reserved. The block of storage is assumed to be appropriately aligned and of the requested size. The address of the created object will not necessarily be the same as that of the block if the object is an array. — end note]

- Amend [expr.new], paragraph 29:

If a new-expression calls a deallocation function, it passes the ~~value returned~~ new-pointer from the allocation function call as the first argument of type `void*`. If a placement deallocation function is called, it is passed the same additional arguments as were passed to the placement allocation function, that is, the same arguments as those specified with the new-placement syntax. If the implementation is allowed to introduce a temporary object or make a copy of any argument as part of the call to the allocation function, it is unspecified whether the same object is used in the call to both the allocation and deallocation functions.

- Amend [expr.delete], paragraph 2:

If the operand has a class type, the operand is converted to a pointer type by calling the above-mentioned conversion function, and the converted operand is used in place of the original operand for the remainder of this subclause. In a single-object delete expression, the value of the operand of `delete` may be a null pointer value, ~~a pointer value that resulted from~~ the new-pointer of a previous non-array *new-expression*, or a pointer to a base class subobject of an object created by such a *new-expression*. If not, the behavior is undefined. In an array delete expression, the value of the operand of `delete` may be a null pointer value or ~~a pointer value that resulted from~~ the new-pointer of a previous array *new-expression*. If not, the behavior is undefined.

[Note 1: This means that the syntax of the delete-expression must match the type of the object allocated by `new`, not the syntax of the *new-expression*. — end note]

[Note 2: A pointer to a `const` type can be the operand of a delete-expression; it is not necessary to cast away the constness (`[expr.const.cast]`) of the pointer expression before it is used as the operand of the delete-expression. — end note]

- Amend `[expr.delete]`, paragraph 7:

If the value of the operand of the delete-expression is not a null pointer value, then:

If the allocation call for the *new-expression* for the object to be deleted was not omitted and the allocation was not extended (`[expr.new]`), the *delete-expression* shall call a deallocation function. The ~~value returned from the allocation call of the new-expression~~ new-pointer shall be passed as the first argument to the deallocation function.

Otherwise, if the allocation was extended or was provided by extending the allocation of another *new-expression*, and the *delete-expression* for every other ~~pointer value produced by a new-expression~~ new-pointer that had storage provided by the extended *new-expression* has been evaluated, the *delete-expression* shall call a deallocation function. The ~~value returned from the allocation call~~ new-pointer of the extended *new-expression* shall be passed as the first argument to the deallocation function.

Otherwise, the *delete-expression* will not call a deallocation function.

[Note 3: The deallocation function is called regardless of whether the destructor for the object or some element of the array throws an exception. — end note]

If the value of the operand of the *delete-expression* is a null pointer value, it is unspecified whether a deallocation function will be called as described above.

- Amend [replacement.functions], paragraph 2:

```
operator new(std::size_t)
operator new(std::size_t, std::align_val_t)
operator new(std::size_t, const std::nothrow_t&)
operator new(std::size_t, std::align_val_t, const std::nothrow_t&)

operator new(std::size_t size, std::return_size_t)
operator new(std::size_t size, std::align_val_t al, std::return_size_t)
operator new(std::size_t size, std::return_size_t, std::nothrow_t)
operator new(std::size_t size, std::align_val_t al, std::return_size_t, s

[...]

operator new[](std::size_t)
operator new[](std::size_t, std::align_val_t)
operator new[](std::size_t, const std::nothrow_t&)
operator new[](std::size_t, std::align_val_t, const std::nothrow_t&)

operator new[](std::size_t size, std::return_size_t)
operator new[](std::size_t size, std::align_val_t al, std::return_size_t)
operator new[](std::size_t size, std::return_size_t, std::nothrow_t)
operator new[](std::size_t size, std::align_val_t al, std::return_size_t, s

[...]
```

- Amend header `<new>` synopsis in [new.syn]:

```
namespace std {
    class bad_alloc;
    class bad_array_new_length;

    struct destroying_delete_t {
        explicit destroying_delete_t() = default;
    };
    inline constexpr destroying_delete_t destroying_delete{};
}
```

```

// global operator new control
struct return_size_t {
    explicit return_size_t() = default;
};

inline constexpr return_size_t return_size{};

template<typename T = void>
struct sized_allocation_t {
    T *ptr;
    size_t bytes;
};

enum class align_val_t : size_t {};

[...]
}

[[nodiscard]] void* operator new(std::size_t size);
[[nodiscard]] void* operator new(std::size_t size, std::align_val_t alignment);
[[nodiscard]] void* operator new(std::size_t size, const std::nothrow_t&) noexcept;
[[nodiscard]] void* operator new(std::size_t size, std::align_val_t alignment,
    const std::nothrow_t&) noexcept;

[[nodiscard]] std::sized_allocation_t<void> ::operator new(
    std::size_t size, std::return_size_t);
[[nodiscard]] std::sized_allocation_t<void> ::operator new(
    std::size_t size, std::align_val_t alignment, std::return_size_t);
[[nodiscard]] std::sized_allocation_t<void> ::operator new(
    std::size_t size, std::return_size_t, std::nothrow_t) noexcept;
[[nodiscard]] std::sized_allocation_t<void> ::operator new(
    std::size_t size, std::align_val_t alignment, std::return_size_t, std::nothrow_t) noexcept;

[...]

[[nodiscard]] void* operator new[](std::size_t size);
[[nodiscard]] void* operator new[](std::size_t size, std::align_val_t alignment);
[[nodiscard]] void* operator new[](std::size_t size, const std::nothrow_t&) noexcept;
[[nodiscard]] void* operator new[](std::size_t size, std::align_val_t alignment,
    const std::nothrow_t&) noexcept;

```



```
[[nodiscard]] std::size_t size, std::return_size_t);
[[nodiscard]] std::size_t size, std::align_val_t alignment, std::return_size_t);
[[nodiscard]] std::size_t size, std::return_size_t, std::nothrow_t) noexcept;
[[nodiscard]] std::size_t size, std::align_val_t alignment, std::return_size_t, std::no
```

[...]

```
[[nodiscard]] void* operator new (std::size_t size, void* ptr) noexcept;
[[nodiscard]] void* operator new[](std::size_t size, void* ptr) noexcept;
void operator delete (void* ptr, void*) noexcept;
void operator delete[](void* ptr, void*) noexcept;
```

- Insert [new.delete.single], after paragraph 4:

```
[[nodiscard]] std::size_t size, std::return_size_t);
[[nodiscard]] std::size_t size, std::align_val_t alignment, std::return_size_t);
```

- Effects: Same as above, except these are called by a placement version of a *new-expression* when a C++ program prefers the size-returning allocation function.
- Replaceable: A C++ program may define functions with either of these function signatures, and thereby displace the default versions defined by the C++ standard library.
- Required behavior: Return a `sized_allocation_t<void>` whose `ptr` member represents the address of a region of N bytes of suitably aligned storage ([basic.stc.dynamic]) for some $N \geq \text{size}$, and whose `bytes` member is N , or else throw a `bad_alloc` exception. This requirement is binding on any replacement versions of these functions.
- Default behavior: Returns `std::size_t size, std::return_size_t)` and `std::size_t size, std::align_val_t alignment, std::return_size_t)` respectively. If a user-provided `operator new` is invoked directly or indirectly, N is `size`.

- Insert [new.delete.single], after paragraph 9:

```
[[nodiscard]] std::size_t allocation_t<void> ::operator new(
    std::size_t size, std::return_size_t, std::nothrow_t) noexcept;
[[nodiscard]] std::size_t allocation_t<void> ::operator new(
    std::size_t size, std::align_val_t alignment, std::return_size_t, std::
```

- Effects: Same as above, except these are called by a placement version of a *new-expression* when a C++ program prefers the size-returning allocation function and a null pointer result as an error indication.
- Replaceable: A C++ program may define functions with either of these function signatures, and thereby displace the default versions defined by the C++ standard library.
- Required behavior: Return a `sized_allocation_t<void>` whose `ptr` member represents the address of a region of *N* bytes of suitably aligned storage ([basic.stc.dynamic]) for some *N* $\geq$ *size*, and whose `bytes` member is *N*, or else return `std::size_t allocation_t<void>{nullptr, 0}`. Each of these `nothrow` versions of `operator new` returns a pointer obtained as if acquired from the (possibly replaced) corresponding non-placement function. This requirement is binding on any replacement versions of these functions.
- Default behavior: Returns `std::size_t allocation_t<void>{operator new(size), N}` and `std::size_t allocation_t<void>{operator new(size, alignment), N}` respectively. If a user-provided `operator new` is invoked directly or indirectly, *N* is *size*. If the call to `operator new` throws, returns `std::size_t allocation_t<void>{nullptr, 0}`.

- Amend [new.delete.single], paragraph 12:

```
void operator delete(void* ptr) noexcept;
void operator delete(void* ptr, std::size_t size) noexcept;
void operator delete(void* ptr, std::align_val_t alignment) noexcept;
void operator delete(void* ptr, std::size_t size, std::align_val_t alignmer
```

[...]

- If the `alignment` parameter is not present, `ptr` shall have been returned by an allocation function without an alignment parameter. If present, the `alignment` argument shall equal the alignment argument passed to the allocation function that returned `ptr`. If present, the `size`

argument shall equal the `size` argument passed to the allocation function that returned `ptr`, if `ptr` was not allocated by a size-returning allocation function. If present, the `size` argument shall satisfy `bytes >= size >= requested` if `ptr` was allocated by a size-returning allocation function, where `bytes` is the size returned in `std::size_allocation_t<void>` and `requested` is the size argument passed to the allocation function.

[...]

- Insert [new.delete.array], after paragraph 4:

```
[[nodiscard]] std::size_allocation_t<void> ::operator new[](
    std::size_t size, std::return_size_t);
[[nodiscard]] std::size_allocation_t<void> ::operator new[](
    std::size_t size, std::align_val_t alignment, std::return_size_t);
```

- *Effects:* Same as above, except these are called by a placement version of a *new-expression* when a C++ program prefers the size-returning allocation function.
- *Replaceable:* A C++ program may define functions with either of these function signatures, and thereby displace the default versions defined by the C++ standard library.
- *Required behavior:* Return a `size_allocation_t<void>` whose `ptr` member represents the address of a region of *N* bytes of suitably aligned storage ([basic.stc.dynamic]) for some *N* $\geq$ *size*, and whose `bytes` member is *N*, or else throw a `bad_alloc` exception. This requirement is binding on any replacement versions of these functions.
- *Default behavior:* Returns `std::size_allocation_t<void>{operator new[](size), N}` and `std::size_allocation_t<void>{operator new[](size, alignment), N}` respectively. If a user-provided operator `new` is invoked directly or indirectly, *N* is *size*.

- Insert [new.delete.array], after paragraph 8:

```
[[nodiscard]] std::size_t allocation_t<void> ::operator new[](
    std::size_t size, std::return_size_t, std::nothrow_t) noexcept;
[[nodiscard]] std::size_t allocation_t<void> ::operator new[](
    std::size_t size, std::align_val_t alignment, std::return_size_t, std::
```

- Effects: Same as above, except these are called by a placement version of a *new-expression* when a C++ program prefers the size-returning allocation function and a null pointer result as an error indication.
- Replaceable: A C++ program may define functions with either of these function signatures, and thereby displace the default versions defined by the C++ standard library.
- Required behavior: Return a `sized_allocation_t<void>` whose `ptr` member represents the address of a region of *N* bytes of suitably aligned storage ([basic.stc.dynamic]) for some *N* $\geq$ *size*, and whose `bytes` member is *N*, or else return `std::size_t allocation_t<void>{nullptr, 0}`. This requirement is binding on any replacement versions of these functions.
- Default behavior: Returns `std::size_t allocation_t<void>{operator new[](size), N}` and `std::size_t allocation_t<void>{operator new[](size, alignment), N}` respectively. If a user-provided operator new is invoked directly or indirectly, *N* is *size*. If the call to `operator new[]` throws, returns `std::size_t allocation_t<void>{nullptr, 0}`.

- Amend [new.delete.array], paragraph 11:

```
void operator delete[](void* ptr) noexcept;
void operator delete[](void* ptr, std::size_t size) noexcept;
void operator delete[](void* ptr, std::align_val_t alignment) noexcept;
void operator delete[](void* ptr, std::size_t size, std::align_val_t alignm
```

[...]

- If the `alignment` parameter is not present, `ptr` shall have been returned by an allocation function without an `alignment` parameter. If present, the `alignment` argument shall equal the `alignment` argument passed to the allocation function that returned `ptr`. If present, the `size` argument shall equal the `size` argument passed to the allocation function that returned `ptr`, if `ptr` was not allocated by a size-returning allocation function. If present, the `size` argument

shall satisfy *bytes* $\geq$ *size* $\geq$ *requested* if *ptr* was allocated by a size-returning allocation function, where *bytes* is the size returned in `std::size_allocation_t<void>` and *requested* is the size argument passed to the allocation function.

[...]

- Amend Table 19 ("Feature-test macros") [cpp.predefined]

Name	Value
<code>__cpp_size_returning_new</code>	PLACEHOLDER DATE

§ 4. Alternative Designs Considered

§ 4.1. Parameters and return types

Another signature we could use would be:

```
enum class return_size_t : std::size_t {};  
void* ::operator new(std::size_t size, std::return_size_t&);
```

(and so on.) This is slightly simpler to read as a signature, but arguably worse in usage:

```
std::tie(obj.ptr, obj.size) = ::operator new(37, std::return_size_t{});  
  
// ...vs...  
  
// Presumably the object implementation wants to contain a size_t,  
// not a return_size_t.  
std::return_size_t rs;  
obj.ptr = ::operator new(37, rs);  
obj.size = rs;
```

More importantly, this form is less efficient. In practice, underlying malloc implementations provide actual definitions of `::operator new` symbols which are called like any other function. Passing a reference parameter requires us to actually return the size via memory.

- Linux ABIs support returning at least two scalar values in registers (even if they're members of a trivially copyable struct) which can be dramatically more efficient.

- The [\[MicrosoftABI\]](#) returns large types by pointer, but this is no worse than making the reference parameter an inherent part of the API.

Whether we use a reference parameter or a second returned value, the interpretation is the same.

§ 4.2. New expressions

§ 4.2.1. Interaction of allocation functions and *new-expressions*

A *new-expression* finds an allocation function by overload resolution. We could in principle entertain a number of new rules for this overload resolution, and we could also consider transforming a non-size-returning allocation function into a size-returning one and vice versa.

- `void* operator new(size_t)` could be converted into a size-returning function by "thunking", returning `{operator new(n), n}`.
- `sized_allocation_t<void> operator new(size_t, return_size_t)` could be converted into a non-size returning function, returning `operator new(n, return_size).ptr`.

This would allow us to consider the following alternative designs:

- Regular `new T` could use a size-returning allocation function if one was available. This would allow users to only provide a single place where they implement a sophisticated allocation function, and it would be used everywhere. We would probably *not* want to change the type of this regular *new-expression*, though.
- Sized `new (std::return_size) T` could fall back to using a non-size-returning allocation function if no size-returning one was available, using thunking. This would make such expressions unconditionally well-formed, but would silently disregard user intent.
- One could even entertain making the type (!) of a *new-expression* depend on the result of overload resolution for the allocation function, but that seems highly novel, brittle, and unhelpful.

None of those extensions seem appealing, since they add complexity and obscure the selection of actual allocation function, so we do not pursue those, and instead always require that a size-returning allocation function be findable in order for a size-returning *new-expression* to be well-formed.

§ 4.2.2. Which kind of "size"

We considered alternatives for returning the size.

- We could return two pointers, the initial object and one past the end of the array (minus the array allocation overhead).

```
auto [start, end] = new (std::return_size) T[5];
for (T* p = start + 5; p != end; p++) {
    new (p) T;
}
for (T* p = start; p != end; p++) {
    p->DoStuff();
}
for (T* p = start + 5; p != end; p++) {
    p->~T();
}
delete[] start;
```

The pair of pointers provides convenience for use with iterator-oriented algorithms. The problem we foresee is that a size-returning allocation function may not provide a size that is an appropriate multiple of `sizeof(T)` (or fail to be a multiple of `alignof(T)`, a possible, but unlikely scenario). This imposes a need for more extensive logic around the *new-expression* in handling the returned allocation size.

- We could return the size in units of `T`, this leads to an inconsistency between the expected usage for `new` and `new[]`:
 - For `new`, we may only end up fitting a single `T` into an allocator size quanta, so the extra space remains unusable. If we can fit multiple `T` into a single allocator size quanta, we now have an array from what was a scalar allocation site. This cannot be foreseen by the compiler as `::operator new` is a replaceable function.
 - For `new[]`, the size in units of `T` can easily be derived from the returned size in bytes.
- We could pass the size in units of `T` or bytes to the constructor of `T`:
 - For `new`, this is especially useful for tail-padded arrays, but neglects default-initialized `T`.
 - For `new[]`, a common use case is expected to be the allocation of arrays of `char`, `int`, etc. The size of the overall array is irrelevant for the individual elements.
- We could return the size via a reference parameter:

```
std::return_end<T> end;
T* p = new (end) T[5];
for (T* p = start + 5; p != end; p++) {
    new (p) T;
```

```

}
for (T* p = start; p != end; p++) {
    p->DoStuff();
}
for (T* p = start + 5; p != end; p++) {
    p->~T();
}

```

or, demonstrated with bytes:

```

std::return_size_t size;
T* p = new (size) T[5];
for (int i = 5; i < size / sizeof(T); i++) {
    new (p[i]) T;
}
for (int i = 0; i < size / sizeof(T); i++) {
    p[i].DoStuff();
}
for (int i = 5; i < size / sizeof(T); i++) {
    p[i].~T();
}
delete[] p;

```

(Casts omitted for clarity.)

As discussed for `::operator new` in [§ 3 Proposed Wording](#), a reference parameter poses difficulties for optimizers and involves returning the size via memory (depending on ABI).

For `new[]` expressions, we considered alternatively initializing the returned `(sz / sizeof(T))` number of elements.

- This would avoid the need to explicitly construct / destruct the elements with the additional returned space (if any).

The *new-initializer* is invoked for the returned number of elements, rather than the requested number of elements. This allows `delete[]` to destroy the correct number of elements (by storing `sz / sizeof(T)` in the array allocation overhead).

- The presented proposal (leaving this space uninitialized) was chosen for consistency with `new`.

§ 5. Discussion

§ 5.1. How many `::operator new`'s?

It is unfortunate that we have so many permutations of `::operator new`--eight seems like far more than we should really need! But there really isn't any significant runtime cost for having them. Use of raw calls to `::operator new` is relatively rare: It's a *building block* for low-level libraries, allocators ([P0401R4]), and so on, so the cognitive burden on C++ users is low.

The authors have considered other alternatives to the additional overloads. At the Jacksonville meeting, EWG suggested looking at parameter packs.

- Parameter packs do not reduce the number of symbols introduced. Implementers still need to provide implementations each of the n overloads.
- Retrofitting parameter packs leaves us with *more* mangled variants. Implementers need to provide both the legacy symbols as well as the parameter pack-mangled symbols.

The authors have also considered APIs where all parameters are passed, thereby requiring a single new overload. This adds further overhead for implementations, as it moves compile-time decisions (is the alignment at or below the minimum guaranteed by `operator new`) into runtime ones.

The alternative to modifying the handling of *new-expressions* invoking deallocation functions (when an exception is thrown) would require additional overloads for `operator delete` / `operator delete[]` whose sole purpose would be to accept and discard the `std::return_size_t`.

§ 5.2. Implementation difficulty

It's worth reiterating that there's a perfectly good trivial implementation of these functions:

```
std::size_allocation_t<void> ::operator new(  
    std::size_t n, std::return_size_t) {  
    return {::operator new(n), n};  
}
```

Malloc implementations are free to properly override this with a more impactful definition, but this paper poses no significant difficulty for toolchain implementers.

Implementation Experience:

- TCMalloc has developed an implementation opensourced on GitHub ([\[MallocExtension\]](#)). While this requires mapping from an integer size class to the true number of bytes, combining this lookup with the allocation is more efficient as we avoid recomputing the sizeclass itself (given a request) or deriving it from the object's address.
- jemalloc is prototyping a `smallocx` function providing a C API for this functionality [\[smallocx\]](#).

§ 5.3. Interaction with Sized Delete

For allocations made with `sized_allocation_t`-returning `::operator new`, we need to relax `::operator delete`'s size argument (16.6.2.1 and 16.6.2.2). For allocations of `T`, the size quanta used by the allocator may not be a multiple of `sizeof(T)`, leading to both the original and returned sizes being unrecoverable at the time of deletion.

Consider the memory allocated by:

```
using T = std::aligned_storage<16, 8>::type;

std::vector<T> v(4);
```

The underlying heap allocation is made with `::operator new(64, std::return_size_t)`.

- The memory allocator may return a 72 byte object: Since there is no `k` such that `sizeof(T) * k = 72`, we can't provide that value to `::operator delete(void*, size_t)`. The only option would be storing 72 explicitly, which would be wasteful.
- The memory allocator may instead return an 80 byte object (5 `T`'s): We now cannot represent the original request when deallocating without additional storage.

For allocations made with

```
std::tie(p, m) = ::operator new(n, std::return_size_t{});
```

we permit `::operator delete(p, s)` where $n \leq s \leq m$.

This behavior is consistent with [\[jemalloc\]](#)'s `sdallocx`, where the deallocation size must fall between the request (`n`) and the actual allocated size (`m`) inclusive.

§ 5.4. Advantages

It's easy to see that this approach nicely solves the problems posed by other methods:

- We pay almost nothing in speed to return an actual-size parameter. For TCMalloc and jemalloc, this is typically a load from to map from *sizeclass* to *size*. This cost is strictly smaller than with `nallocx` or the like, as that same translation must be performed in addition to the duplicative work previously discussed.
- We are told exactly the size we have, without risk of UB. We can avoid subsequent reallocations when growing to a buffer to an already-allocated size.
- Allocator telemetry knows actual request sizes exactly.

§ 5.5. Naming

The library support type is named `sized_allocation_t`, based on LEWG's naming suggestions and for consistency (in choice of `_t`) with the other allocation library support types (`size_t`, `align_val_t`, `nothrow_t`, etc.). We expect this to be spelled rarely.

Its members are `ptr` and `bytes`. `ptr` for the memory returned is an intuitive name. `bytes` conveys units in its name (`bytes`). Since new expressions can return additional storage under this proposal, this distinguishes it from returning the amount of storage available for *whole* objects. This contrasts with [P0401R4], which does return storage for whole objects and whose support type's field is named `count`.

§ 6. Related work

[P0401R4] considers this problem at the level of the `Allocator` concept. Ironically, the lack of the above API was one significant problem in its original revision: how could an implementation of `std::allocator` provide the requested feedback in a way that would work with any underlying malloc implementation?

§ 7. History

§ 7.1. R9 → R10

A detailed design of *new-expressions* has been added. Previously, changes to *new-expressions* had only been outlined at a high level. As a result, previously suggested changes to overload resolution rules for

finding the allocation function (which was allowed to fall back to a non-size-returning allocation function, with unclear consequences) have been removed. The relation between *new-expressions* and allocation functions is now described explicitly.

Sections are reorganized: alternative designs are now separate from a discussion of the chosen design, and an expanded section on *new-expressions* has been moved up before the proposed wording.

§ 7.2. R8 → R9

CWG reviewed [\[P0901R8\]](#) via [\[CWG2021Telecon\]](#).

Wording changes have been made to the proposal based on CWG feedback and assistance from Ryan McDougall.

§ 7.3. R7 → R8

LEWG reviewed [\[P0901R6\]](#) via telecon.

Poll: Send P0901R6, after changing `p` to `ptr` and `n` to `bytes` in `sized_allocation_t`, to electronic ballot to be forwarded to CWG.

SF	F	N	A	SA
4	11	0	0	0

- Applied member name feedback from LEWG to rename `sized_allocation_t`'s members from `p` to `ptr` and `n` to `bytes`. Added naming rationale section.
- Update reference to revised [\[P0401R4\]](#).
- Refined wording in proposal.

§ 7.4. R6 → R7

- Removed extraneous `std::` typo

§ 7.5. R5 → R6

LEWG reviewed [\[P0901R5\]](#) library support types at [\[Prague\]](#).

- LEWG discussed the similarity to `std::span`. Participants in the discussion were concerned about the potential library layering issue (`<new>` layers below practically everything) and that ownership is conveyed in the return value. In contrast, `std::span` is non-owning.
- LEWG discussed whether the type needed a name. It needs to be named to allow users to replace `operator new` and spell the type.

LEWG took an approval poll of different names

5	sized_ptr_t
2	sized_ptr
3	memblock_t
7	memblock
0	memspan_t
2	memspan
8	alloc_size_t
4	alloc_size
5	alloc_t
0	alloc
13	sized_allocation_t
17	sized_allocation
0	alloc_span_t
2	alloc_span
9	allocation_t
9	allocation
16	alloc_result_t
14	alloc_result
12	allocation_result_t
16	allocation_result

Based on this poll, LEWG directed the paper authors to consider `sized_allocation`, `alloc_result_t`, and `allocation_result`.

LEWG additionally polled on whether the type should have a `_t` suffix.

SF	F	N	A	SA
0	6	7	8	1

Based on this feedback, the authors have chosen `sized_allocation_t`, based on LEWG's naming suggestions and for consistency (in choice of `_t`) with the other allocation library support types (`size_t`, `align_val_t`, `nothrow_t`, etc.). We expect this to be spelled rarely.

§ 7.6. R4 → R5

EWG reviewed P0901R4 at [\[Cologne\]](#).

Poll: P0901R4 as presented, forward to LEWG for C++23, *not* C++20.

SF	F	N	A	SA
2	11	14	2	0

- Fixed typos in examples.
- Added proposed feature test macro.
- Added LEWG audience for library support type names.

§ 7.7. R3 → R4

- Update reference to revised [\[P0401R1\]](#).

§ 7.8. R2 → R3

- Added proposed wording.
- For newly added allocation functions, `std::nothrow_t` is passed by value, rather than reference.

§ 7.9. R1 → R2

Applied feedback from San Diego Mailing

- Moved from passing `std::return_size_t` parameter by reference to by value. For many ABIs, this is more optimizable and to the authors' knowledge, no worse on any other.
- Added rationale for not using parameter packs for this functionality.

§ 7.10. R0 → R1

Applied feedback from [\[JacksonvilleMinutes\]](#).

- Clarified in [§ 3 Proposed Wording](#) the desire to leverage the existing "replacement functions" wording of the IS, particularly given the close interoperation with the existing `::operator new/::operator delete` implementations.
- Added a discussion of the Microsoft ABI in [§ 3 Proposed Wording](#).
- Noted in [§ 5.1 How many ::operator new's?](#) the possibility of using a parameter pack.
- Added a proposal for [§ 2.2 New-expressions](#), as requested by EWG.

Additionally, a discussion of [§ 5.3 Interaction with Sized Delete](#) has been added.

§ References

§ Informative References

[Cologne]

Size feedback in operator new. 2019-09-17.

[CppDraft]

Working Draft, Standard for Programming Language C++. 2022-04-12. URL: <https://eel.is/c++draft/>

[CWG2021Telecon]

Paper Review Teleconference 2021-03-22. 2021-03-22.

[JacksonvilleMinutes]

Jacksonville 2018 minutes. 2018-03-15.

[JEMALLOC]

jemalloc(3) - Linux man page. URL: <http://jemalloc.net/jemalloc.3.html>

[MallocExtension]

TCMalloc Malloc Extensions. URL: https://github.com/google/tcmalloc/blob/master/tcmalloc/malloc_extension.h

[MicrosoftABI]

Return Values. 2016-11-03. URL: <https://docs.microsoft.com/en-us/cpp/build/return-values-cpp>

[P0401R1]

Jonathan Wakely; Chris Kennelly. *Providing size feedback in the Allocator interface*. 2019-06-11. URL: <http://wg21.link/P0401R1>

[P0401R4]

Jonathan Wakely; Chris Kennelly. *Providing size feedback in the Allocator interface*. 2020-11-14.
URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p0401r4.html>

[P0901R5]

Size feedback in operator new. 2019-10-06. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0901r5.html>

[P0901R6]

Size feedback in operator new. 2020-03-01. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p0901r6.html>

[P0901R8]

Chris Kennelly, Andrew Hunter. *Size feedback in operator new*. 15 December 2020. URL: <https://wg21.link/p0901r8>

[Prague]

LEWG Prague Minutes for P901R5. 2020-02-10.

[SizedDelete]

Lawrence Crowl. *C++ Sized Deallocation*. URL: <http://wg21.link/n3536>

[SMALLOCX]

Add experimental API to support P0901r0. URL: <https://github.com/jemalloc/jemalloc/pull/1270>

[TCMalloc]

TCMalloc. URL: <https://github.com/google/tcmalloc>